Modelling Software-based Systems

Lecture 5 The access control problem in Event-B

Telecom Nancy (3A LE)

Dominique Méry
Telecom Nancy,Université de Lorraine

13 octobre 2026

dominique.mery@loria.fr

Summary

1. Refinement of models
2. Summary on Event-B
3. Case Study The Access Control (J.-R. Abrial)
4. Conclusion

Current Section Summary

1. Refinement of models
2. Summary on Event-B
3. Case Study The Access Control (J.-R. Abrial)
4. Conclusion

Summing up...

- Refinement relates Event-B models
- Problem for starting a refinement-based development
- Problem for finding the best abstract model
- Problem for discharging unproved proof obligations generated for each refinement step
- The Access Control Problem

Current Section Summary

1. Refinement of models
2. Summary on Event-B
3. Case Study The Access Control (J.-R. Abrial)
4. Conclusion

Events as Relations over Variables Values

- Each variable V has a current value v, a next value v'
- Each event e over variables V is defined by a relation over v and v' denoted $BA(e)(v, v')$.
- An event e has local parameters, variables, guards and actions.
- Events *observe* changes over state variables and changes can be related to code execution or to physical phenomena.

Simple Form of an Event

- An event of the simple form is denoted by :

$< event_name > \widehat{=}$
WHEN
$< condition >$
THEN
$< action >$
END

where
- $< event_name >$ is an identifier
- $< condition >$ is the firing condition of the event
- $< action >$ is a generalized substitution (parallel "assignment")

Non-deterministic Form of an Event

- An event of the non-deterministic form is denoted by :

$< event_name > \widehat{=}$
ANY $< variable >$ **WHERE**
$< condition >$
THEN
$< action >$
END

where
- $< event_name >$ is an identifier
- $< variable >$ is a (list of) variable(s)
- $< condition >$ is the firing condition of the event
- $< action >$ is a generalized substitution (parallel "assignment")

Shape of a Generalized Substitution

A generalized substitution can be

- Simple assignment : $x := E$
- Generalized assignment : $x : |P(x, x')$
- Set assignment : $x :\in S$
- Parallel composition : $\begin{array}{l} T \\ \cdots \\ U \end{array}$

,

Invariant Preservation Verification (0)

$$\text{INVARIANT} \ \wedge \ \text{GUARD}$$
$$\Longrightarrow$$
$$\text{ACTION establishes INVARIANT}$$

Invariant Preservation Verification (1)

- Given an event of the simple form :

EVENT e $\widehat{=}$
WHEN
$G(x)$
THEN
$x := E(x)$
END

and invariant $I(x)$ to be preserved, the statement to prove is :

$$I(x) \quad \wedge \quad G(x) \quad \Longrightarrow \quad I(E(x))$$

Invariant Preservation Verification (2)

- Given an event of the simple form :

EVENT e $\widehat{=}$
WHEN
$G(x)$
THEN
$x :| P(x, x')$
END

and invariant $I(x)$ to be preserved, the statement to prove is :

$$I(x) \quad \wedge \quad G(x) \quad \wedge \quad P(x, x') \quad \Longrightarrow \quad I(x')$$

Invariant Preservation Verification (3)

- Given an event of the simple form :

EVENT e $\widehat{=}$
WHEN
$G(x)$
THEN
$x :\in S(x)$
END

and invariant $I(x)$ to be preserved, the statement to prove is :

$$I(x) \quad \wedge \quad G(x) \quad \wedge \quad x' \in S(x) \quad \Longrightarrow \quad I(x')$$

Invariant Preservation Verification (4)

- Given an event of the non-deterministic form :

EVENT e $\widehat{=}$
  ANY v **WHERE**
    $G(x, v)$
  THEN
    $x := E(x, v)$
  END

and invariant $I(x)$ to be preserved, the statement to prove is :

$$I(x) \quad \wedge \quad G(x, v) \quad \Longrightarrow \quad I(E(x, v))$$

Refinement Technique (1)

- Abstract models works with variables x, and concrete one with y
- A gluing invariant $J(x, y)$ links both sets of vrbls
- Each abstract event is refined by concrete one (see below)

Refinement Technique (2)

- Some new events may appear : they refine “skip”
- Concrete events must not block more often than the abstract ones
- The set of new event alone must always block eventually

Correct Refinement Verification (1)

- Given an abstract and a corresponding concrete event

EVENT ae $\widehat{=}$
WHEN
$G(x)$
THEN
$x := E(x)$
END

EVENT ce $\widehat{=}$
WHEN
$H(y)$
THEN
$y := F(y)$
END

and invariants $I(x)$ and $J(x, y)$, the statement to prove is :

$$I(x) \ \wedge \ J(x, y) \ \wedge \ H(y) \implies G(x) \ \wedge \ J(E(x), F(y))$$

Correct Refinement Verification (1)

- Given an abstract and a corresponding concrete event

EVENT ae $\widehat{=}$
WHEN
$G(x)$
THEN
$x := E(x)$
END

EVENT ce $\widehat{=}$
WHEN
$H(y)$
THEN
$y := F(y)$
END

and invariants $I(x)$ and $J(x, y)$, the statement to prove is :

$$I(x) \ \wedge \ J(x, y) \ \wedge \ H(y) \implies G(x) \ \wedge \ J(E(x), F(y))$$

- $BA(ae)(x, x') \ \widehat{=} \ G(x) \wedge x' = E(x)$
- $BA(ce)(y, y') \ \widehat{=} \ H(y) \wedge y' = F(y)$

Correct Refinement Verification (2)

- Given an abstract and a corresponding concrete event

EVENT ae $\widehat{=}$
ANY v **WHERE**
$G(x, v)$
THEN
$x := E(x, v)$
END

EVENT ce $\widehat{=}$
ANY w **WHERE**
$H(y, w)$
THEN
$y := F(y, w)$
END

$$I(x) \quad \wedge \quad J(x, y) \quad \wedge \quad H(y, w)$$
$$\Longrightarrow$$
$$\exists v \cdot (G(x, v) \quad \wedge \quad J(E(x, v), F(y, w)))$$

- $BA(ae)(x, x') \ \widehat{=} \ \exists v.G(x, v) \wedge x' = E(x)$
- $BA(ce)(y, y') \ \widehat{=} \ \exists w.H(y, w) \wedge y' = F(y)$

Correct Refinement Verification (3)

- Given a NEW event

EVENT ne $\widehat{=}$
  WHEN
    $H(y)$
  THEN
    $y := F(y)$
  END

and invariants $I(x)$ and $J(x, y)$, the statement to prove is :

$$I(x) \quad \wedge \quad J(x, y) \quad \wedge \quad H(y) \quad \Longrightarrow \quad J(x, F(y))$$

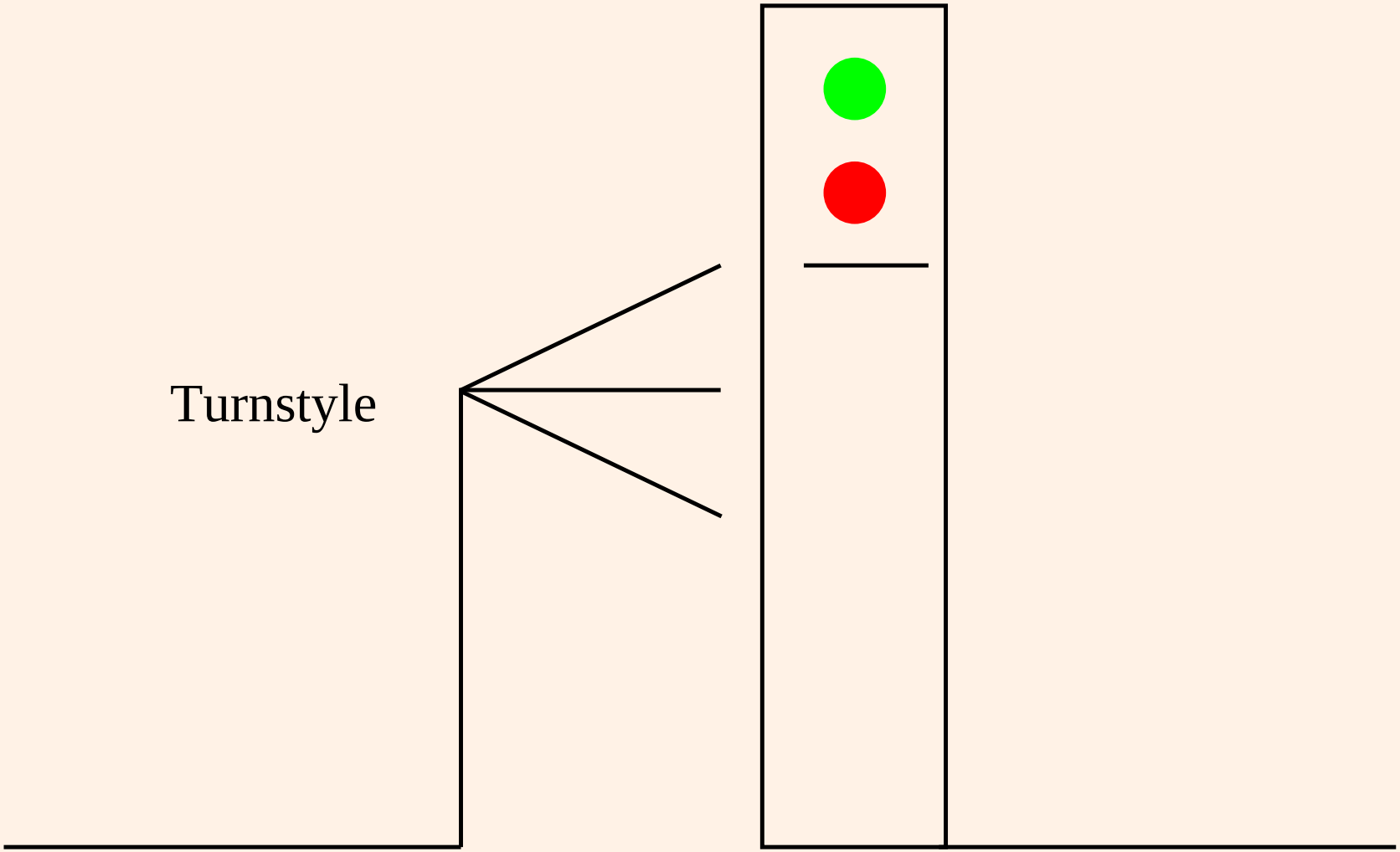
- $BA(ne)(y, y') \ \widehat{=} \ H(y) \wedge y' = E(y)$

Current Section Summary

1. Refinement of models
2. Summary on Event-B
3. Case Study The Access Control (J.-R. Abrial)
4. Conclusion

A Case Study by J.-R. Abrial

- To control accesses into locations.
- People are assigned certain authorizations
- Each person is given a magnetic card
- Doors are “one way” turnstyles
- Each turnstyle is equipped with :
    - a card reader
    - two lights (one green, the other red)


Turnstyle

Access Protocol (after introducing card in reader)

- If access permitted
  - green light turned on
  - turnstyle unblocked for 30 sec
- Passing, or 30 sec elapsed
  - green light turned off
  - turnstyle blocked again
- If access refused
  - red light turned on for 2 sec
  - turnstyle stays blocked

Goal of System Study

- Sharing between Control and Equipment
- For this : constructing a closed model
- Defining the physical environment
- Possible generalization of problem
- Studying safety questions
- Studying synchronisation questions
- Studying marginal behaviour

Basic System Properties

P1 : The model concerns people and locations
P2 : A person is authorized to be in some locations
P3 : A person can only be in one location at a time
D1 : Outside is a location where everybody can be
P4 : A person is always in some location
P5 : A person is always authorized to be in his location

Example

Sets

$$\begin{aligned} \text{persons} &= \{\, \text{p1}, \text{p2}, \text{p3} \,\} \\ \text{locations} &= \{\, \text{l1}, \text{l2}, \text{l3}, \text{l4} \,\} \end{aligned}$$

Authorizations

p1	l2, l4
p2	l1, l3, l4
p3	l2, l3, l4

Correct scenario

p1	l4
p2	l4
p3	l4

$\rightarrow$

p1	l2
p2	l4
p3	l4

$\rightarrow$

p1	l2
p2	l1
p3	l4

$\rightarrow$

p1	l4
p2	l1
p3	l4

$\rightarrow$

p1	l4
p2	l1
p3	l3

Model (1)

Basic sets : persons P and locations B (prop. P1)
Constant : authorizations A (prop. P2)
A is a binary relation between P and B

$$\mathrm{A} \in \mathrm{P} \leftrightarrow \mathrm{B}$$

Model (2)

Constant : $outside$ is a location where everybody is authorized to be (decision D1)

$$outside \in \mathrm{B}$$

$$\mathrm{P} \times \{outside\} \subseteq \mathrm{A}$$

Model (3)

Variable : situations c (prop. P3 and P4)
c is a total function between P and B
A total function is a special case of a binary relation

$$\mathrm{c} \in \mathrm{P} \rightarrow \mathrm{B}$$

Invariant : situations compatible with auth. (prop. P5)
The function c is included in the relation A

$$\mathrm{c} \subseteq \mathrm{A}$$

A magic event which can be observed

- GUARD : $\begin{cases} \text{- Given some person } p \text{ and location } l \\ \text{- } p \text{ is authorized to be in } l : p, l \in \mathrm{A} \\ \text{- } p \text{ is not currently in } l : \mathrm{c}(p) \neq l \end{cases}$
- ACTION : - p jumps into l

EVENT observation1 $\widehat{=}$
  ANY p, l **WHERE**
    $p \in \mathrm{P} \quad \wedge$
    $l \in \mathrm{B} \quad \wedge$
    $p \mapsto l \in \mathrm{A} \quad \wedge$
    $\mathrm{c}(p) \neq l$
  THEN
    $\mathrm{c}(p) := l$
  END

Relation overriding

Given two relations a and b

Overriding a by b yields a new relation $a \lhd\!\!+ b$

$$a \lhd\!\!+ b \quad \widehat{=} \quad (\mathsf{dom}\,(b) \lhd a) \ \cup \ b$$

Abbreviation

$$f(x) := y \quad \widehat{=} \quad f := f \lhd\!\!+ \{x \mapsto y\}$$

Invariant Preservation Proof

INVARIANT $\wedge$ GUARD

$\Longrightarrow$

ACTION establishes INVARIANT

$$
\begin{array}{l}
\mathrm{C} \subseteq \mathrm{A} \quad \wedge \\
p \in \mathrm{P} \quad \wedge \\
l \in \mathrm{B} \quad \wedge \\
p \mapsto l \in \mathrm{A} \\
\Longrightarrow \\
(\{p\} \lhd \mathrm{C}) \cup \{p \mapsto l\} \subseteq \mathrm{A}
\end{array}
$$

First Refinement : Introducing Geometry

P6 : The geometry define how locations communicate
P7 : A location does not communicate with itself
P8 : Persons move between communicating locations

Refined Model

Constant : communication $\mathrm{STRUCTURE}$ (prop. P6 and P7)
$\mathrm{STRUCTURE}$ is a binary relation between B
The intersection of $\mathrm{STRUCTURE}$ with the identity relation on B is empty

$$\mathrm{STRUCTURE} \in \mathrm{B} \leftrightarrow \mathrm{B}$$

$$\mathrm{STRUCTURE} \cap \mathsf{id}(\mathrm{B}) = \varnothing$$

Correct Refinement Verification (reminder)

Concrete events do not block more often than abstract ones

$I(x) \quad \wedge \quad J(x,y) \quad \wedge$
disjunction of abstract guards
$\Longrightarrow$
disjunction of concrete guards

New events block eventually (decreasing the same quantity $V(y)$)

$$I(x) \quad \wedge \quad J(x,y) \quad \wedge \quad H(y) \quad \wedge \quad V(y) = n \implies V(F(y)) < n$$

Refined Event

Event (prop. P8)
The guard is strengthened
The current location of p and the new location l must communicate

EVENT observation1 $\widehat{=}$
  ANY p, l **WHERE**
    $p \in \mathrm{P} \quad \wedge$
    $l \in \mathrm{B} \quad \wedge$
    $p \mapsto l \in \mathrm{A} \quad \wedge$
    $\mathrm{C}(p) \neq l$
  THEN
    $\mathrm{C}(p) := l$
  END

EVENT observation2 $\widehat{=}$
  REFINES $observation1$
  ANY p, l **WHERE**
    $p \in \mathrm{P} \quad \wedge$
    $l \in \mathrm{B} \quad \wedge$
    $p \mapsto l \in \mathrm{A} \quad \wedge$
    $\mathrm{C}(p) \mapsto l \in \mathrm{STRUCTURE}$
  THEN
    $\mathrm{C}(p) := l$
  END

Proofs

Invariant preservation : Success

Guard strengthening : Success

$$\exists\,(p,l)\cdot(\,p\mapsto l\ \in\ \mathrm{A}\ \wedge\ \mathrm{C}(p)\mapsto l\ \in\ \mathrm{STRUCTURE}\,)$$
$$\Rightarrow$$
$$\exists\,(p,l)\cdot(\,p\mapsto l\ \in\ \mathrm{A}\ \wedge\ \mathrm{C}(p)\neq l\,)$$

Deadlockfreeness : Failure

$$\exists\,(p,l)\cdot(\,p\mapsto l\ \in\ \mathrm{A}\ \wedge\ \mathrm{C}(p)\neq l\,)$$
$$\Rightarrow$$
$$\exists\,(p,l)\cdot(\,p\mapsto l\ \in\ \mathrm{A}\ \wedge\ \mathrm{C}(p)\mapsto l\ \in\ \mathrm{STRUCTURE}\,)$$

Safety Problem

P9 : No person must remain blocked in a location.

Solution

P10 : Any person authorized to be in a location must also be authorized to go in another location which communicates with the first one.

$$\boxed{\mathrm{A} \subseteq \mathrm{A} ; \textsc{structure}^{-1}}$$

$$p \mapsto l \in \mathrm{A} \implies \exists m \cdot (p \mapsto m \in \mathrm{A} \ \wedge \ l \mapsto m \in \textsc{structure})$$

Example

p1	l2	p2	l4
p1	l4	p3	l2
p2	l1	p3	l3
p2	l3	p3	l4

A

l1	l3
l1	l4
l3	l2
l4	l1
l4	l2
l4	l3

STRUCTURE

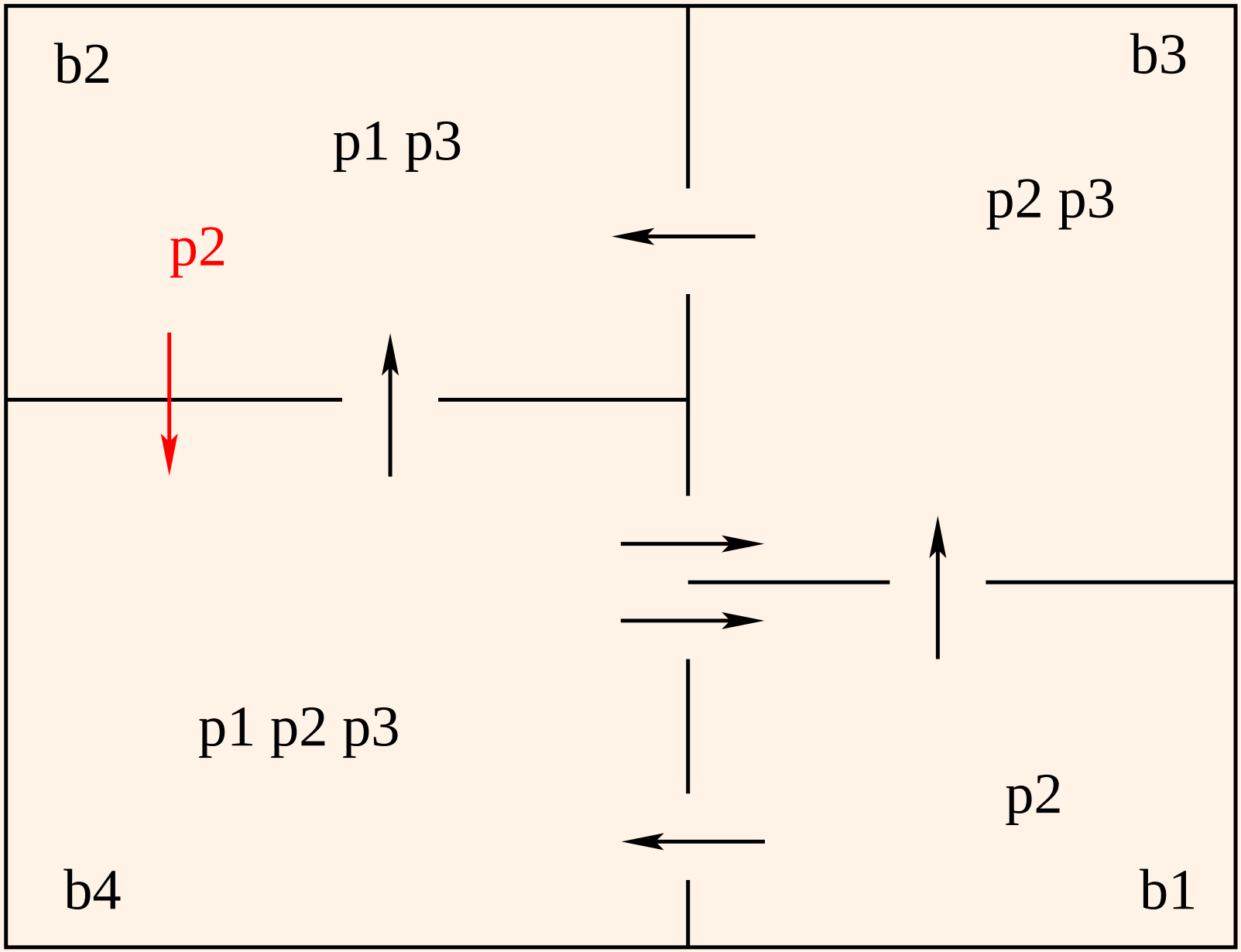
l1	l4
l2	l3
l2	l4
l3	l1
l3	l4
l4	l1

STRUCTURE^{-1}

p1	l1
p1	l3
p1	l4
p2	l1

A; STRU

- Opening a door between l2 and l4
- Authorizing p2 to go to l2


b2
b3
p1 p3
p2 p3
p2
p1 p2 p3
p2
b4
b1

Solution

p1	l2	p2	l4
p1	l4	p3	l2
p2	l1	p3	l3
p2	l2	p3	l4
p2	l3		

A

l1	l3
l1	l4
l2	l4
l3	l2
l4	l1
l4	l2
l4	l3

STRUCTURE

l1	l4
l2	l3
l2	l4
l3	l1
l3	l4
l4	l1
l4	l2

STRUCTURE^{-1}

p1	l1	
p1	l2	
p1	l3	
p1	l4	
p2	l1	
p2	l2	

A; STRUCTU

Decision

D2 : The system that we are going to construct does not guarantee that people can move “outside”.

A better solution (1)

Constante : $exit$ is a function, included in com, with no cycle

$$exit \in \mathrm{B} - \{outside\} \to \mathrm{B}$$

$$exit \subseteq com$$

$$\forall s \cdot (s \subseteq \mathrm{B} \implies (s \subseteq exit^{-1}[s] \implies s = \varnothing))$$

$$\forall x \cdot (x \in s \implies \exists y \cdot (y \in s \ \wedge \ (x, y) \in exit))$$
$$\implies$$
$$s = \varnothing$$

$exit$ is a tree spanning the graph represented by com

A better solution (2)

P10' : Every person authorized to be in a location (which is not "outside") must also be authorized to be in another location communicating with the former and leading towards the exit.

$$\boxed{\mathrm{A} \rhd \{outside\} \subseteq \mathrm{A} ; exit^{-1}}$$

$$\begin{array}{l} p \mapsto l \in \mathrm{A} \ \wedge \\ l \neq outside \\ \Longrightarrow \\ p \mapsto exit(l) \in \mathrm{A} \end{array}$$

For the experts

Show that no cycle implies the possibility to prove property by induction and vice-versa

$$\forall s \cdot (s \subseteq \mathrm{B} \ \wedge \ s \subseteq exit^{-1}[s] \implies s = \varnothing)$$
$$\Leftrightarrow$$
$$\forall t \cdot (\ t \subseteq \mathrm{B} \ \wedge \ outside \in t \ \wedge \ exit^{-1}[t] \subseteq t \implies t = \mathrm{B})$$

$$t \subseteq \mathrm{B}$$
$$outside \in t$$
$$\forall (x, y) \cdot ((x \mapsto y) \in exit \ \wedge \ y \in t \implies x \in t)$$
$$\implies$$
$$t = \mathrm{B}$$

Second Refinement : Introducing Doors

P11 : Locations communicate via one-way doors.

P12 : A person get through a door only if accepted.

P13 : A door is acceptable by at most one person at a time.

P14 : A person is accepted for at most one door only.

P15 : A person is accepted if at the origin of the door.

P16 : A person is accepted if authorized at destination.

Extending the Model (1)

Set : the set DOORS of doors
Constants : The origin ORG and destination DST of a door (prop. P11)

$$\mathrm{ORG} \in \mathrm{DOORS} \to \mathrm{B}$$
$$\mathrm{DST} \in \mathrm{DOORS} \to \mathrm{B}$$
$$\mathrm{STRUCTURE} = (\mathrm{ORG}^{-1} ; \mathrm{DST})$$

Extending the Model (2)

Variable : the rel. DAP between persons and doors (prop. P12 to P16)

$$\begin{array}{l} \mathrm{DAP} \in \mathrm{P} \rightarrowtail\!\!\!\!\!\!\rightarrow \mathrm{DOORS} \\ (\mathrm{DAP} ; \mathrm{ORG}) \subseteq \mathrm{C} \\ (\mathrm{DAP} ; \mathrm{DST}) \subseteq \mathrm{A} \end{array}$$

Second Refinement : More Properties

P17 : Green light of a door is lit when access is accepted.
P18 : When a person has got through, the door blocks.
P19 : After 30 seconds, the door blocks automatically.
P20 : Red light is lit for 2 sec.when access is refused.
P21 : Red and green lights are not lit simultaneously.

Extending the Model (3)

Definition : GREEN is exactly the range of DAP (prop. P17 to P19)

$$\text{GREEN} \;\widehat{=}\; \text{ran}\,(\text{DAP})$$

Extending the Model (4)

Variable : The set red of red doors (prop. P20)

$$red \subseteq \text{DOORS}$$

Invariant : GREEN and red are incompatible (prop. P21)

$$\text{GREEN} \cap red = \varnothing$$

Condition for Admission

P22 : Person p is accepted through door d if

- p is situated within the origin of d
- p is authorized to move to the dest. of d
- p is not engaged with another door

$$\begin{array}{l}\mathsf{admitted}\,(p,d) \;\widehat{=}\\ \quad \mathrm{ORG}(d) = \mathrm{C}(p) \;\wedge\\ \quad p \mapsto \mathrm{DST}(d) \in \mathrm{A} \;\wedge\\ \quad p \notin \mathsf{dom}\,(dap)\end{array}$$

A New Event (1)

Accepting a person p - GUARD :

- Given some person p and door d
- d is neither green nor red
- p is admissible through d

- ACTION : - make p authorized to pass d

EVENT accept $\widehat{=}$
  ANY p, d **WHERE**
    $p \in \mathrm{P} \quad \wedge$
    $d \in \mathrm{DOORS} \quad \wedge$
    $d \notin \mathrm{GREEN} \cup red \quad \wedge$
    admitted (p, d)
  THEN
    $\mathrm{DAP}(p) := d$
  END

A New Event (2)

Refusing a person p

- GUARD :
  - Given some person p and door d
  - d is neither green nor red
  - p is not admissible through d
- ACTION :
  - lit the red light

EVENT refuse $\widehat{=}$
  ANY p, d **WHERE**
    $p \in \mathrm{P} \quad \wedge$
    $d \in \mathrm{DOORS} \quad \wedge$
    $d \notin \mathrm{GREEN} \cup red \quad \wedge$
    $\neg$ admitted (p, d)
  THEN
    $red := red \cup \{d\}$
  END

Refining Event OBSERVATION2

EVENT observation2 $\widehat{=}$
  ANY p, l **WHERE**
    $p \in \mathrm{P}$
    $l \in \mathrm{B}$
    $p, l \in \mathrm{A}$
    $\mathrm{C}(p) \mapsto l \in \mathrm{STRUCTURE}$
  THEN
    $\mathrm{C}(p) := l$
  END

EVENT observation3 $\widehat{=}$
  REFINES observation2
  ANY d **WHERE**
    $d \in \mathrm{GREEN}$
  THEN
    $\mathrm{C}(\mathrm{DAP}^{-1}(d)) := \mathrm{DST}(d)$
    $\mathrm{DAP} := \mathrm{DAP} \rhd \{d\}$
  END

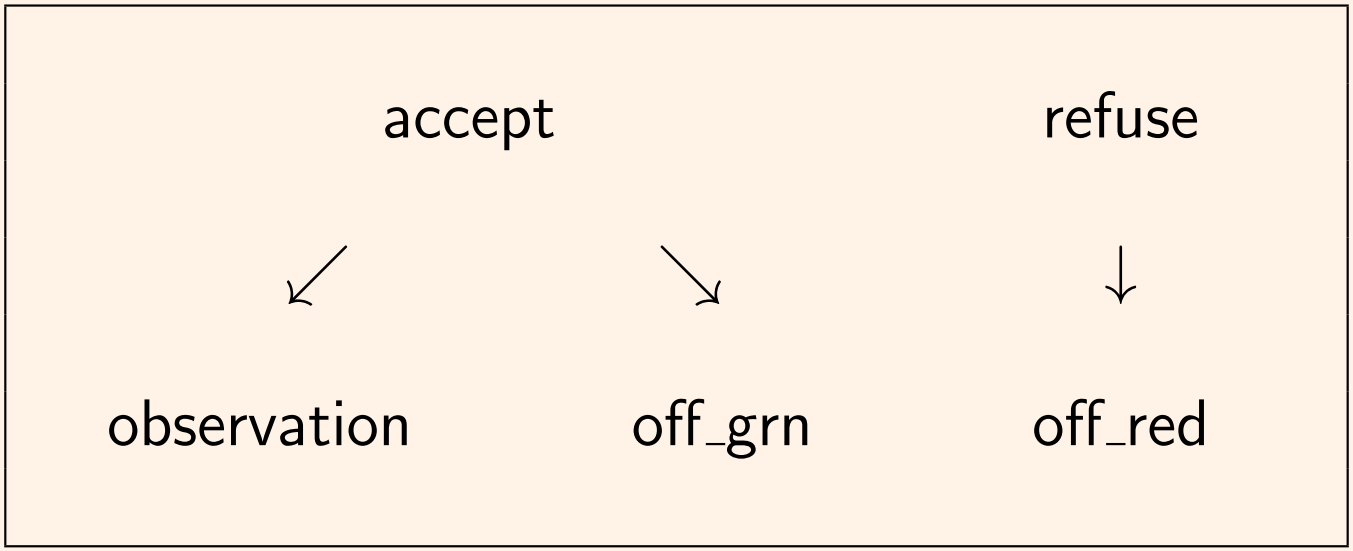
New Event (3)

Turning lights off

EVENT off_grn $\widehat{=}$
ANY d **WHERE**
$d \in \text{GREEN}$
THEN
$\text{DAP} := \text{DAP} \rhd \{d\}$
END

EVENT off_red $\widehat{=}$
ANY d **WHERE**
$d \in red$
THEN
$red := red - \{d\}$
END

Synchronization



Proofs

- Event observation is a correct refinement : OK
- Other events refine skip : OK
- Event observation does not deadlock more : OK
- New events do not take control indefinitely : FAILURE

Permanent Obstruction of Card Readers

DANGER

- People without the required authorizations
  try indefinitely to enter some locations.
- Other people with the authorization
  always change mind at the last moment.

SOLUTIONS

- Make such practice impossible ? ? ?
- Card Readers can “swallow” a card

Final Decision

D3 : The system we are going to construct will not prevent people from blocking doors indefinitely :

- either by trying indefinitely to enter places into which they are not authorized to enter,
- or by indefinitely abandoning "on the way" their intention to enter the places in which they are in fact authorized to enter¨.

Third Refinement : Introducing Card Readers

A decision

D4 : Each card reader is supposed to stay blocked between :

- the sending of a card to the system
- the reception of an acknowledgement.

Third Refinement : Model Extension

The set BLR of blocked Card Readers
The set $mCard$ of messages sent by Card Readers
The set $mAckn$ of acknowledgment messages

$$BLR \subseteq \text{DOORS}$$

$$mCard \in \text{DOORS} \pfun \text{P}$$

$$mAckn \subseteq \text{DOORS}$$

Third Refinement : Invariant

$\mathsf{dom}\,(mCard)$, GREEN, red, $mAckn$ partition BLR

$$\mathsf{dom}\,(mCard) \cup \text{GREEN} \cup red \cup mAckn = BLR$$

$$\mathsf{dom}\,(mCard) \cap (\text{GREEN} \cup red \cup mAckn) = \varnothing$$

$$mAckn \cap (\text{GREEN} \cup red) = \varnothing$$

Events (1)

EVENT CARD $\widehat{=}$
  ANY p, d
  WHERE
    $p \in \mathrm{P}$
    $d \in \mathrm{DOORS} - BLR$
  THEN
    $BLR := BLR \cup \{d\}$
    $mCard := mCard \cup \{d \mapsto p\}$
  END

Events (2)

EVENT accept3 $\widehat{=}$
  ANY p, d
  WHERE
    $p \in \mathrm{P}$
    $d \in \mathrm{DOORS}$
    $d \notin \mathrm{GREEN} \cup red$
    admitted (p, d)
  THEN
    $\mathrm{DAP}(p) := d$
  END

EVENT accept4 $\widehat{=}$
  REFINES accept3
  ANY p, d
  WHERE
    $d \mapsto p \in mCard$
    admitted (p, d)
  THEN
    $\mathrm{DAP}(p) := d$
    $mCard := mCard - \{d \mapsto p\}$
  END

Events (3)

EVENT refuse4 $\widehat{=}$
  REFINES refuse3
  ANY p, d
  WHERE
    $d \mapsto p \in mCard$
    $\neg$ admitted (p, d)
  THEN
    $red := red \cup \{d\}$
    $mCard := mCard - \{d \mapsto p\}$
  END

Events (4)

EVENT observation4 $\widehat{=}$
  REFINES observation3
  ANY d
  WHERE
    $d \in \text{GREEN}$
  THEN
    $\text{C}(\text{DAP}^{-1}(d)) := \text{DST}(d)$
    $\text{DAP} := \text{DAP} \rhd \{d\}$
    $mAckn := mAckn \cup \{d\}$
  END

Events (5)

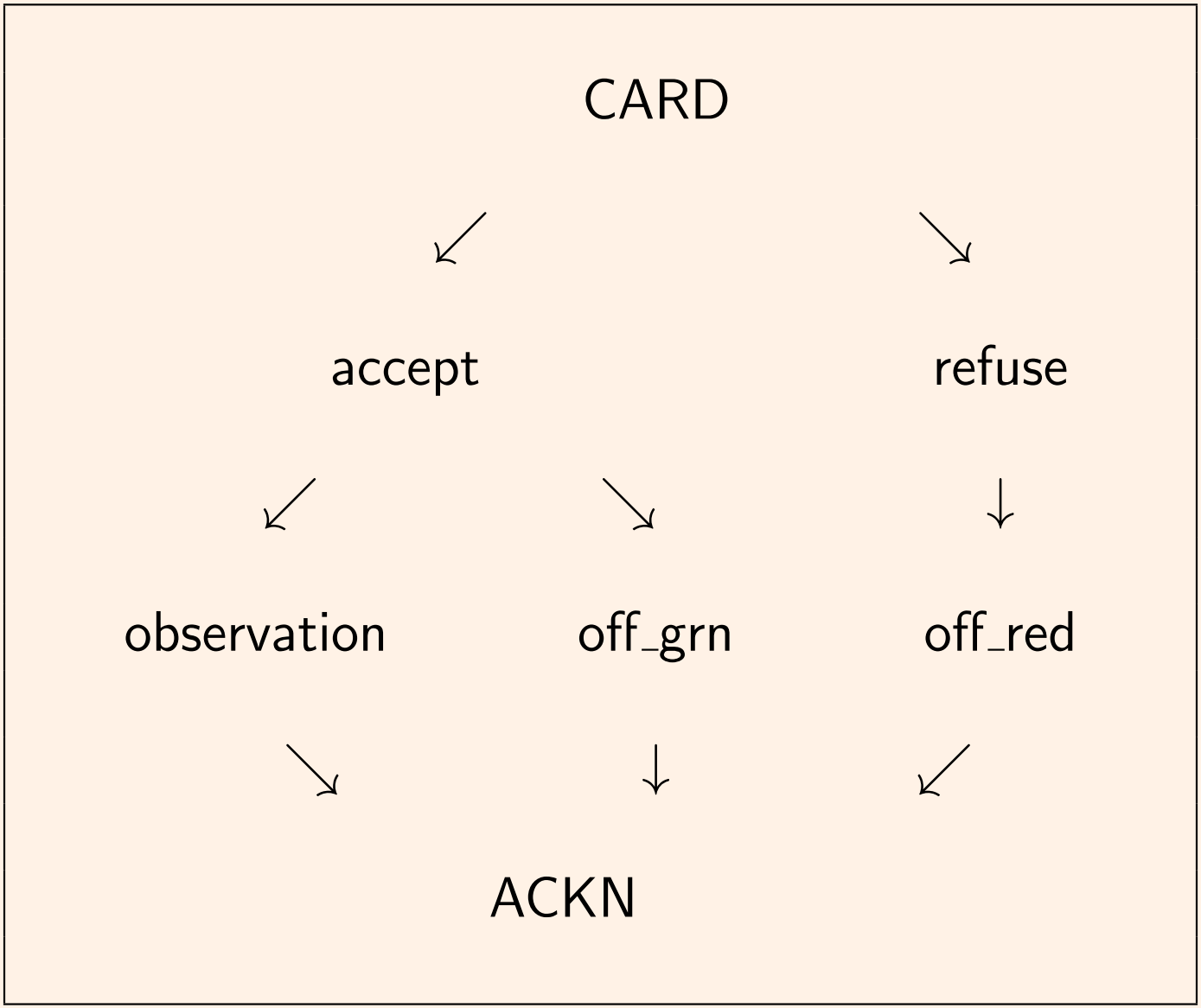
EVENT off_grn $\widehat{=}$
ANY d **WHERE**
$d \in \text{GREEN}$
THEN
$\text{DAP} := \text{DAP} \rhd \{d\}$
$mAckn := mAckn \cup \{d\}$
END

EVENT off_red $\widehat{=}$
ANY d **WHERE**
$d \in red$
THEN
$red := red - \{d\}$
$mAckn := mAckn \cup \{d\}$
END

Events (6)

EVENT ACKN $\widehat{=}$
  ANY d **WHERE**
    $d \in mAckn$
  THEN
    $BLR := BLR - \{d\}$
    $mAckn := mAckn - \{d\}$
  END

Synchronization



Fourth Refinement : Physical Doors and Lights

Decisions

D5 : When a door has been cleared, it blocks itself automatically
without any intervention from the control system.

D6 : Each door incorporates a local clock for
- the extinction of the green light after 30 sec.
- the extinction of the red light after 2 sec.

Extending the Model : the Green Chain (1)

The set $mAccept$ of acceptance messages (to doors)
The set GRN of physical green doors
The set $mPass$ of passing messages (from doors)
The set $mOff_grn$ of messages (from doors)

$$mAccept \subseteq \text{DOORS}$$

$$GRN \subseteq \text{DOORS}$$

$$mPass \subseteq \text{DOORS}$$

$$mOff_grn \subseteq \text{DOORS}$$

Extending the Model : the Green Chain (2)

$mAccept, GRN, mPass, mOff_grn$ partition GREEN

$$mAccept \cup GRN \cup mPass \cup mOff_grn = \text{GREEN}$$

$$mAccept \cap (GRN \cup mPass \cup mOff_grn) = \varnothing$$

$$GRN \cap (mPass \cup mOff_grn) = \varnothing$$

$$mPass \cap mOff_grn = \varnothing$$

Extending the Model : the Red Chain (1)

The set $mRefuse$ of messages (to doors)
The set RED of phyical red doors
The set $mOff_red$ of messages (from doors)

$$mRefuse \subseteq \text{DOORS}$$

$$RED \subseteq \text{DOORS}$$

$$mOff_red \subseteq \text{DOORS}$$

Extending the Model : the Red Chain (2)

$mRefuse, RED, mOff_red$ partition red

$$mRefuse \cup RED \cup mOff_red = red$$

$$mRefuse \cap (RED \cup mOff_red) = \varnothing$$

$$RED \cap mOff_red = \varnothing$$

Events (1)

EVENT accept $\widehat{=}$
  ANY p, d **WHERE**
    $d \mapsto p \in mCard \quad \wedge$
    admitted (p, d)
  THEN
    $\mathrm{DAP}(p) := d$
    $mCard := mCard - \{d \mapsto p\}$
    $mAccept := mAccept \cup \{d\}$
  END

Events (2)

EVENT ACCEPT $\widehat{=}$
  ANY d **WHERE**
    $d \in mAccept$
  THEN
    $GRN := GRN \cup \{d\}$
    $mAccept := mAccept - \{d\}$
  END

Events (3)

EVENT PASS $\widehat{=}$
  ANY d **WHERE**
    $d \in GRN$
  THEN
    $GRN := GRN - \{d\}$
    $mPass := mPass \cup \{d\}$
  END

Events (4)

EVENT observation5 $\widehat{=}$
REFINES observation4 **ANY** d **WHERE**
$d \in mPass$
THEN
$\mathrm{C}(\mathrm{DAP}^{-1}(d)) := \mathrm{DST}(d)$
$\mathrm{DAP} := \mathrm{DAP} \rhd \{d\}$
$mAckn := mAckn \cup \{d\}$
$mPass := mPass - \{d\}$
END

Events (5)

EVENT OFF_GRN $\widehat{=}$
  ANY d **WHERE**
    $d \in GRN$
  THEN
    $GRN := GRN - \{d\}$
    $mOff_grn := mOff_grn \cup \{d\}$
  END

Events (6)

EVENT off_grn $\widehat{=}$
  ANY d **WHERE**
    $d \in mOff_grn$
  THEN
    $\mathrm{DAP} := \mathrm{DAP} \rhd \{d\}$
    $mAckn := mAckn \cup \{d\}$
    $mOff_grn := mOff_grn - \{d\}$
  END

Events (7)

EVENT refuse $\widehat{=}$
  ANY p, d **WHERE**
    $d \mapsto p \in mCard \quad \wedge$
    $\neg$ admitted (p, d)
  THEN
    $red := red \cup \{d\}$
    $mCard := mCard - \{d \mapsto p\}$
    $mRefuse := mRefuse \cup \{d\}$
  END

Events (8)

EVENT REFUSE $\widehat{=}$
  ANY d **WHERE**
    $d \in mRefuse$
  THEN
    $RED := RED \cup \{d\}$
    $mRefuse := mRefuse - \{d\}$
  END

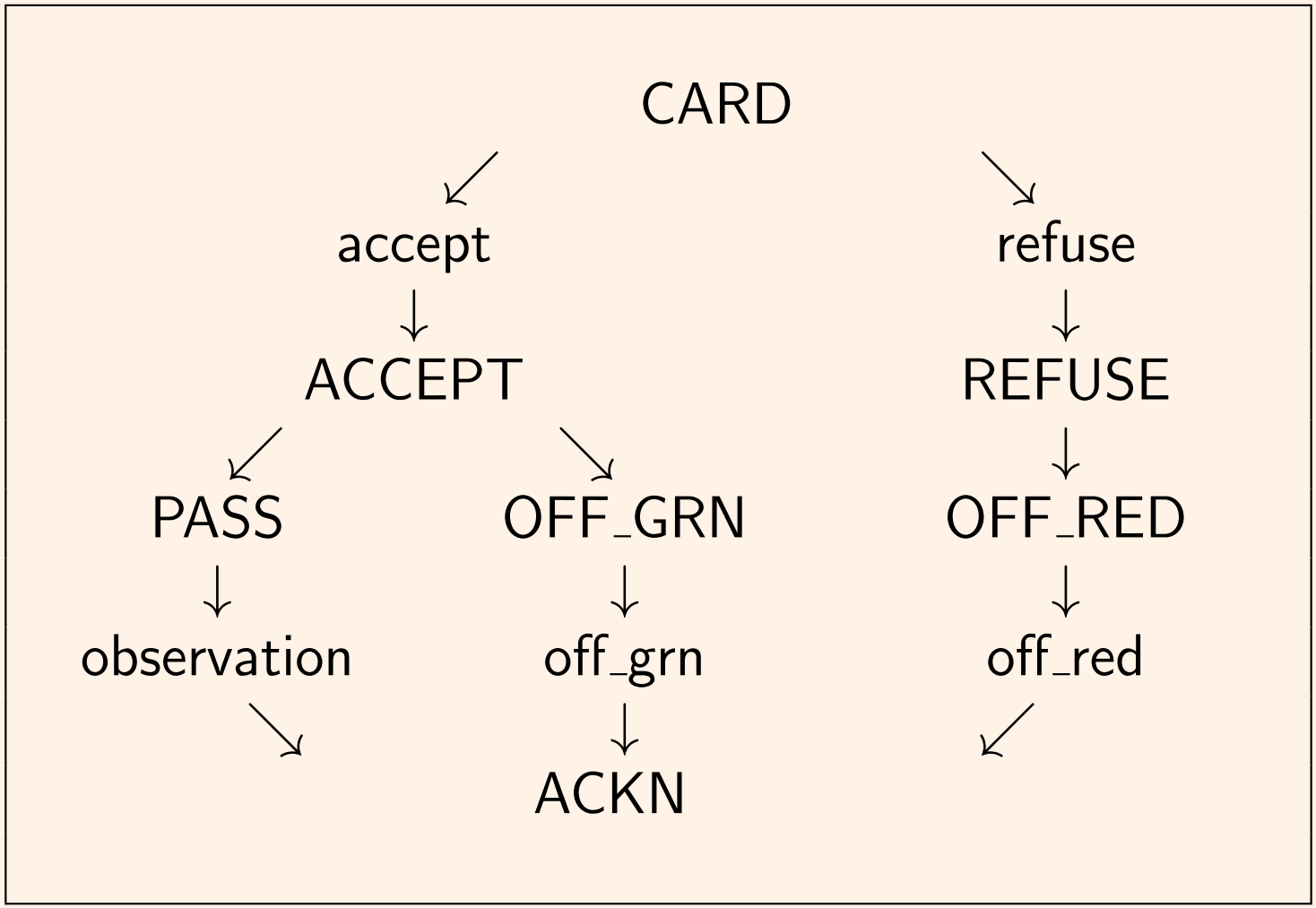
Events (9)

EVENT OFF_RED $\widehat{=}$
  ANY d **WHERE**
    $d \in RED$
  THEN
    $RED := RED - \{d\}$
    $mOff_red := mOff_red \cup \{d\}$
  END

Events (10)

EVENT off_red $\widehat{=}$
  ANY d **WHERE**
    $d \in mOff_red$
  THEN
    $red := red - \{d\}$
    $mAckn := mAckn \cup \{d\}$
    $mOff_red := mOff_red - \{d\}$
  END

Synchronization



Communications

Hardware	Network			Software
CARD	$\rightarrow$	$mCard$	$\rightarrow$	$\begin{cases} \text{accept (1)} \\ \text{refuse (2)} \end{cases}$
ACCEPT	$\leftarrow$	$mAccept$	$\leftarrow$	(1)
PASS	$\rightarrow$	$mPass$	$\rightarrow$	observation (3)
OFF_GRN	$\rightarrow$	$mOff_grn$	$\rightarrow$	off_grn (3)
REFUSE	$\leftarrow$	$mRefuse$	$\leftarrow$	(2)
OFF_RED	$\rightarrow$	$mOff_red$	$\rightarrow$	off_red (3)
ACKN	$\leftarrow$	$mAckn$	$\leftarrow$	(3)

Decomposition (1)

Software Data

$$
\begin{array}{l}
aut \in \mathrm{P} \leftrightarrow \mathrm{B} \\
\mathrm{ORG} \in \textsc{doors} \rightarrow \mathrm{B} \\
\mathrm{DST} \in \textsc{doors} \rightarrow \mathrm{B} \\
\mathrm{A} \subseteq \mathrm{A}; \mathrm{DST}^{-1}; \mathrm{ORG} \\
\mathrm{C} \in \mathrm{P} \rightarrow \mathrm{B} \\
\\
dap \in \mathrm{P} \rightarrowtail \textsc{doors} \\
red \subseteq \textsc{doors}
\end{array}
$$

Decomposition (2)

Network data

$$mCard \in \text{DOORS} \pfun \mathrm{P}$$

$$mAckn \subseteq \text{DOORS}$$

$$mAccept \subseteq \text{DOORS}$$

$$mPass \subseteq \text{DOORS}$$

$$mOff_grn \subseteq \text{DOORS}$$

$$mRefuse \subseteq \text{DOORS}$$

$$mOff_red \subseteq \text{DOORS}$$

Decomposition (3)

"Physical" Data

$$BLR \subseteq \text{DOORS}$$

$$GRN \subseteq \text{DOORS}$$

$$RED \subseteq \text{DOORS}$$

Software Operations

EVENT test_soft(p, d)

EVENT accept_soft(p, d)

EVENT refuse_soft(d)

EVENT pass_soft(d)

EVENT off_grn_soft(d)

EVENT off_red_soft(d)

Physical Operations

$(p, d) \longleftarrow$ CARD_HARD

ACCEPT_HARD(d)

REFUSE_HARD(d)

$d \longleftarrow$ PASS_HARD

$d \longleftarrow$ OFF_GRN_HARD

$d \longleftarrow$ OFF_RED_HARD

ACKN_HARD(d)

Network Software Operations

$(p, d) \longleftarrow$ read_card

write_accept(d)

write_refuse(d)

$d \longleftarrow$ read_pass

$d \longleftarrow$ read_off_grn

$d \longleftarrow$ read_off_red

write_ackn(d)

Network Physical Operations

SEND_CARD(p, d)

d $\longleftarrow$ RCV_ACCEPT

d $\longleftarrow$ RCV_REFUSE

SEND_PASS(d)

SEND_OFF_GRN(d)

SEND_OFF_RED(d)

d $\longleftarrow$ RCV_ACKN

EVENT CARD $\widehat{=}$
  VAR p, d **IN**
    $(p, d) \longleftarrow$ READ_HARD;
    SEND_CARD(p, d)
  END

EVENT accept_refuse $\widehat{=}$
  VAR p, d, b **IN**
    $(p, d) \longleftarrow$ read_card;
    $b \longleftarrow$ **EVENT** test_soft(p, d);
    IF $b =$ true **THEN** **EVENT** accept_soft(p, d); write_accept(d)
    ELSE **EVENT** refuse_soft(d); write_refuse(d) **END**
  END

EVENT ACCEPT $\widehat{=}$
  VAR d **IN**
    $d \longleftarrow$ RCV_ACCEPT;
    ACCEPT_HARD(d)
  END

EVENT REFUSE $\widehat{=}$
  VAR d **IN**
    d $\longleftarrow$ RCV_REFUSE;
    REFUSE_HARD(q)
  END

EVENT PASS $\widehat{=}$
VAR d **IN**
$d \longleftarrow$ PASS_HARD;
SEND_PASS(d)
END

EVENT OFF_GRN $\widehat{=}$
VAR d **IN**
$d \longleftarrow$ OFF_GRN_HARD;
SEND_OFF_GRN(d)
END

EVENT OFF_RED $\widehat{=}$
VAR d **IN**
$d \longleftarrow$ OFF_RED_HARD;
SEND_OFF_RED(d)
END

EVENT observation $\widehat{=}$
VAR d **IN**
$d \longleftarrow$ read_pass;
EVENT pass_soft(d);
write_ackn(d)
END

EVENT off_grn $\widehat{=}$
VAR d **IN**
$d \longleftarrow$ read_off_grn;
EVENT off_grn_soft(d);
write_ackn(d)
END

EVENT off_red $\widehat{=}$
VAR d **IN**
$d \longleftarrow$ read_off_red;
EVENT off_red_soft(d);
write_ackn(d)
END

EVENT ACKN $\widehat{=}$
VAR d **IN**
d $\longleftarrow$ RCV_ACKN;
ACKN_HARD(d)
END

Conclusion

22 Properties et 6 “System” Decisions - One Problem Generalization
- Access between locations
- One Negative Choice :
- Possible Card Readers Obstructions
- Three Physical Decisions
- Automatic Blocking of Doors
- Automatic Blocking of Card Readers
- Setting up of Clocks on Doors
- The overall development required 183 proofs
- 147 automatic (80%)
- 36 interactive

Current Section Summary

1. Refinement of models
2. Summary on Event-B
3. Case Study The Access Control (J.-R. Abrial)
4. Conclusion

Conclusion

- Identify an abstract model
- Identify constants and states
- Identify components
- Plan the refinement
- Start as long as the model is not well defined !

Generalization of the Access Control Problem

- A is a variable which can be modified by events mdeolling the administration of the access control model :
  - adding authorizations to a set of persons
  - removing or deleting authorizations of a set of persons
- Generalizing to other problems :
  - a set of users U has access to a set of resources R.
  - a set of rooms R is managed by a set of keycards K.
  - a set of users U has access to a set of services S.